

PingGuard

Distributed Uptime Monitoring System

Complete Technical Architecture & Build Blueprint

System Design · Chapter-by-Chapter Engineering Specification ·
End-to-End Execution Roadmap

Stack: FastAPI · Pydantic · SQLAlchemy 2.0 (async) · Alembic · Celery · Redis · httpx · PostgreSQL ·
Docker Compose

Prepared as a principal-architect-level engineering reference. Every chapter maps directly to a shippable milestone.

Table of Contents

1. Overall System Architecture & Macro Flowchart

3

1.1 System Overview

3

1.2 Macro Flowchart

3

2. Chapter-by-Chapter Detailed Blueprint

6

Chapter 1 — FastAPI & Async Python — The Request Layer

6

Chapter 2 — SQLAlchemy & Alembic — The Persistence Layer

11

Chapter 3 — Celery & Redis — Distributed Task Execution

15

Chapter 4 — Celery Beat — The Scheduling Heartbeat

19

Chapter 5 — httpx & Network Resilience

23

Chapter 6 — Docker & Docker Compose — Orchestration

27

3. End-to-End Execution Roadmap

32

3.0 Project Scaffold

32

3.1 Milestone Sequence

32

3.2 Beyond Local Development

34

1. Overall System Architecture & Macro Flowchart

1.1 System Overview

PingGuard has exactly two "sides" that run at very different speeds, and almost every design decision in this blueprint exists to keep them from blocking each other.

The **fast side** is the FastAPI application. It only ever does quick, bounded work: parse a request, validate it with Pydantic, read or write a row in PostgreSQL through SQLAlchemy's async engine, and return JSON. Nothing in this path is allowed to make an outbound network call to a website the user is monitoring — that call could take anywhere from 5 milliseconds to a 5-second timeout, and doing it inline would stall the event loop for every other client sharing that worker.

The **slow side** is everything related to actually reaching out to the internet: pinging URLs that might be slow, unreachable, or actively hostile (self-referencing loops, huge response bodies, expired certificates). This work is deliberately pushed onto a completely separate process pool — Celery workers — that communicate with the API only through Redis, never through a direct function call. If a worker hangs for 30 seconds pinging a dead server, it affects that one worker slot, not the API's ability to serve the next request.

The bridge between the two sides is **Celery Beat**, a single lightweight scheduler process that wakes up on a fixed interval, asks PostgreSQL "which monitors are due right now?", and drops one task per due monitor onto the Redis queue. Beat never pings anything itself and never blocks on network I/O — its only job is a fast database query and a fast enqueue.

End-to-end, a single monitor's life cycle looks like this: a client registers a URL (validated, persisted) → Beat's next tick notices it is due → a task lands on Redis → a free worker picks it up, pings the URL with strict timeouts, and classifies the result → the result is written back to PostgreSQL → the client's next status poll reads that row through the same async API path it used to create the monitor. Docker Compose is the only component that "knows about" all five moving pieces (API, Postgres, Redis, worker, Beat) at once — everything else only knows about its immediate neighbor in the pipeline, which is what makes each piece independently scalable and independently replaceable.

1.2 Macro Flowchart

The diagram below traces a monitor from creation through its first scheduled check and back to a client read — crossing every subsystem covered in Chapters 1 through 6.

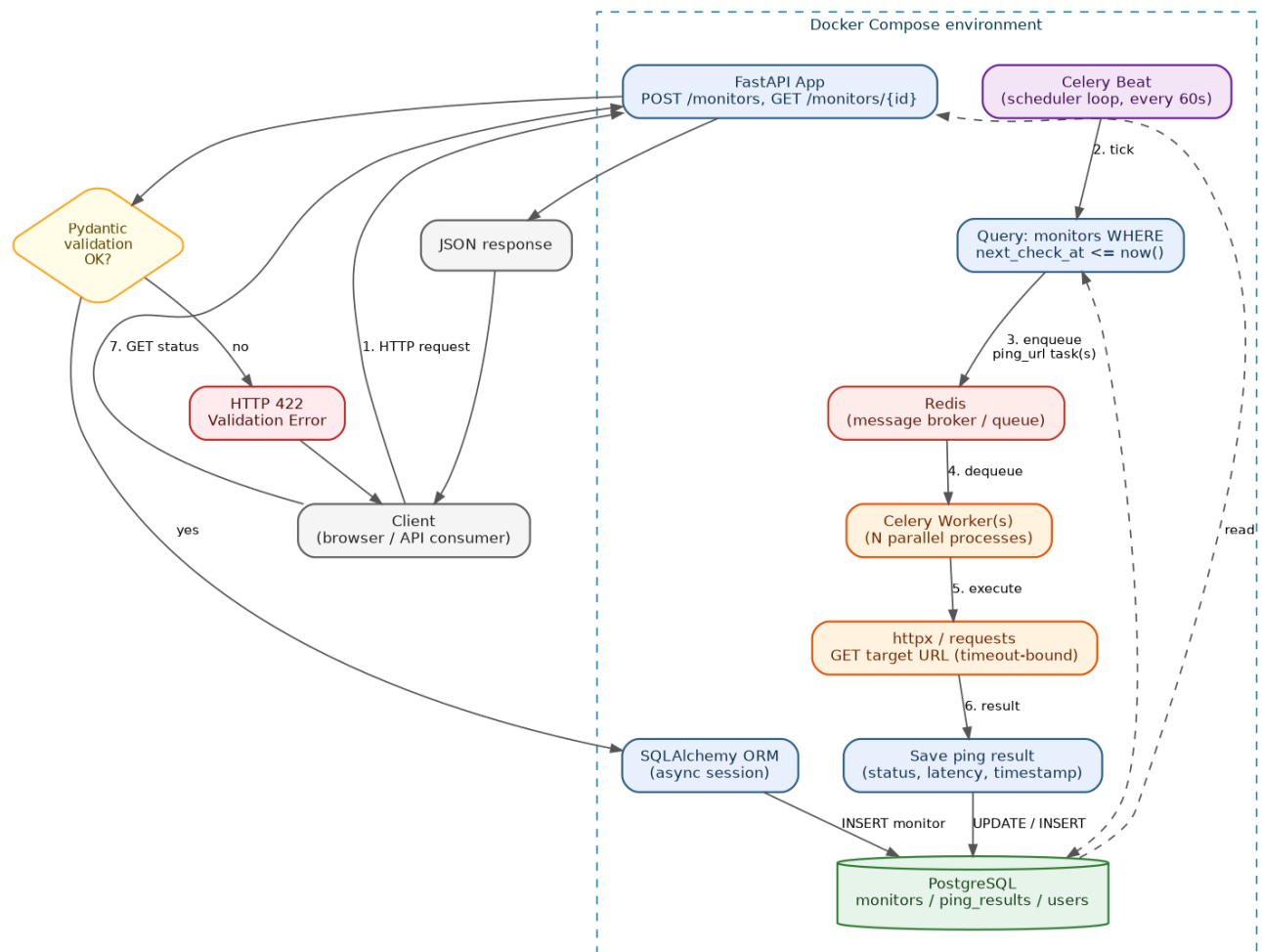


Figure 1.1 — Top-level system flowchart: request path (left) and scheduled ping path (right), unified inside the Docker Compose environment.

Mermaid.js source (paste into any Mermaid-compatible renderer / Markdown file to reproduce this diagram):

```

flowchart TB
    Client["Client (browser / API consumer)"]
    subgraph Docker ["Docker Compose environment"]
        API["FastAPI App<br/>POST /monitors, GET /monitors/{id}"]
        Valid{"Pydantic validation OK?"}
        Err["HTTP 422 Validation Error"]
        ORMWrite["SQLAlchemy ORM (async session)"]
        PG["PostgreSQL<br/>monitors / ping_results / users"]
        Beat["Celery Beat (scheduler loop, every 60s)"]
        Query["Query: monitors WHERE next_check_at <= now()"]
        Redis["Redis (message broker / queue)"]
        Worker["Celery Worker(s) (N parallel processes)"]
        Httpx["httpx / requests GET target URL (timeout-bound)"]
        Save["Save ping result (status, latency, timestamp)"]
    end
    end
    Client -->|"1. HTTP request"| API
    API --> Valid
    Valid -->|no| Err --> Client
    Valid -->|yes| ORMWrite -->|"INSERT monitor"| PG
    Beat -->|"2. tick"| Query
    Query --> PG
    Query -->|"3. enqueue ping_url task(s)"| Redis
    Redis -->|"4. dequeue"| Worker
    Worker -->|"5. execute"| Httpx
    Httpx -->|"6. result"| Save
    Save -->|"UPDATE / INSERT"| PG
    Client -->|"7. GET status"| API
    PG -->|read| API
    API --> Client

```

2. Chapter-by-Chapter Detailed Blueprint

Each subsection below is self-contained: objective and scope, a dedicated flowchart, the exact libraries and schemas involved, the runtime execution pipeline, key code, and the specific edge cases that break naive implementations of this pattern in production.

Chapter 1 — FastAPI & Async Python — The Request Layer

A. Chapter Objective & Scope

Inputs:

- Raw HTTP request bodies (JSON) from clients creating, updating, or querying monitors
- Path/query parameters (monitor IDs, pagination cursors, filters)

Output Artifacts:

- Validated Python objects (Pydantic models) handed to the persistence layer (Chapter 2)
- JSON HTTP responses with correct status codes (201, 200, 404, 422)
- OpenAPI schema (auto-generated) for client SDK generation and interactive docs at /docs

Core Goals:

- Never block the event loop on network or disk I/O
- Reject malformed input before it reaches business logic or the database
- Keep request handling decoupled from ping execution (that belongs to Chapter 3)

B. Chapter Flowchart

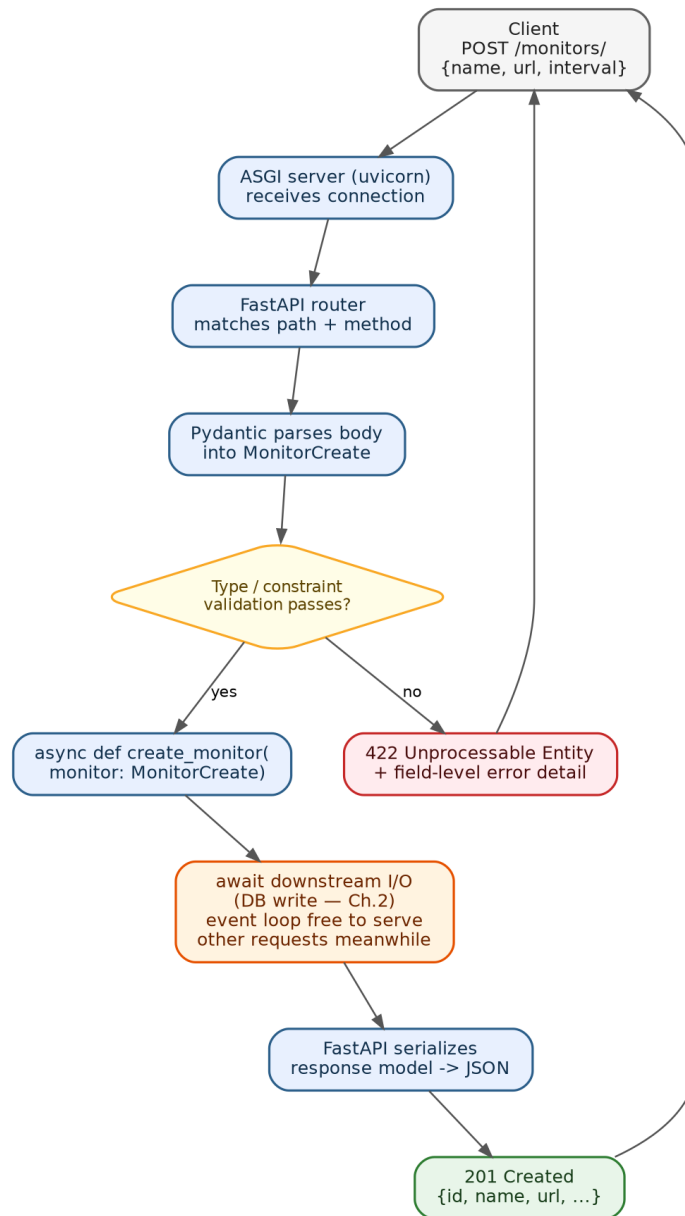


Figure 2.1 — Step-by-step component interaction for Chapter 1.

Mermaid.js source (paste into any Mermaid-compatible renderer / Markdown file to reproduce this diagram):

```

flowchart TB
    Client["Client: POST /monitors/ {name, url, interval}"]
    ASGI["ASGI server (uvicorn) receives connection"]
    Route["FastAPI router matches path + method"]
    Pyd["Pydantic parses body into MonitorCreate"]
    Valid{"Type / constraint validation passes?"}
    Err["422 Unprocessable Entity + field-level error detail"]
    Handler["async def create_monitor(monitor: MonitorCreate)"]
    Await["await downstream I/O (DB write - Ch.2)<br/>event loop free to serve other requests"]
    Ser["FastAPI serializes response model -> JSON"]
    Resp["201 Created {id, name, url, ...}"]

    Client --> ASGI --> Route --> Pyd --> Valid
    Valid -->|no| Err --> Client
    Valid -->|yes| Handler --> Await --> Ser --> Resp --> Client

```

C. Code-Side & Technical Implementation

Libraries, Frameworks & Tools

Library / Tool	Exact Role in PingGuard
fastapi	ASGI web framework; routing, dependency injection, OpenAPI generation
uvicorn[standard]	ASGI server that actually runs the async event loop in production
pydantic v2	Request/response schema validation and serialization
pydantic-settings	Typed environment-variable configuration (DB URL, Redis URL, etc.)
python-multipart	Required by FastAPI for form-encoded/file uploads, if added later

Data Structures & Schemas

Every endpoint is backed by explicit Pydantic models — never raw dicts. This gives you validation, serialization, and OpenAPI docs for free from the same class.

```
from pydantic import BaseModel, HttpUrl, Field, ConfigDict
from datetime import datetime
from enum import Enum

class MonitorStatus(str, Enum):
    UP = "up"
    DEGRADED = "degraded"
    DOWN = "down"
    PENDING = "pending"

class MonitorCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    url: HttpUrl
    check_interval_seconds: int = Field(default=60, ge=15, le=86400)

class MonitorRead(BaseModel):
    model_config = ConfigDict(from_attributes=True) # allows .model_validate(orm_obj)
    id: int
    name: str
    url: HttpUrl
    check_interval_seconds: int
    status: MonitorStatus
    last_checked_at: datetime | None
    next_check_at: datetime | None

class MonitorUpdate(BaseModel):
    name: str | None = None
    check_interval_seconds: int | None = Field(default=None, ge=15, le=86400)
```

Execution Pipeline

- 1 uvicorn accepts the TCP connection and hands the request to FastAPI's ASGI app callable
- 2 The router matches method + path against registered routes (O(1) trie lookup, not linear scan)
- 3 FastAPI extracts the request body and instantiates the declared Pydantic model
- 4 Pydantic runs field-level validation (type coercion, constraints like ge=15) and raises ValidationError on failure — FastAPI converts this automatically into an HTTP 422
- 5 On success, the endpoint coroutine runs; any 'await' inside it (DB call, Redis call) yields control back to the event loop so other requests keep being served concurrently
- 6 The return value is validated against response_model and serialized to JSON
- 7 Middleware (CORS, logging, request-ID injection) wraps the whole cycle

Key Code Snippets

Dependency-injected DB session per request

```
from fastapi import Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession

async def get_db() -> AsyncSession:
    async with SessionLocal() as session:
        yield session    # closed automatically when the request ends, even on exception

@app.post("/monitors/", response_model=MonitorRead, status_code=status.HTTP_201_CREATED)
async def create_monitor(payload: MonitorCreate, db: AsyncSession = Depends(get_db)):
    monitor = Monitor(**payload.model_dump())
    db.add(monitor)
    await db.commit()
    await db.refresh(monitor)
    return monitor

@app.get("/monitors/{monitor_id}", response_model=MonitorRead)
async def get_monitor(monitor_id: int, db: AsyncSession = Depends(get_db)):
    monitor = await db.get(Monitor, monitor_id)
    if monitor is None:
        raise HTTPException(status_code=404, detail="Monitor not found")
    return monitor
```

Never do this — blocking call inside an async route

```
# BAD: requests.get() is synchronous and blocks the ENTIRE event loop,
# freezing every other concurrent request on this worker for the full timeout.
@app.get("/bad-example")
async def bad_example():
    import requests
    r = requests.get("https://example.com", timeout=10)    # <-- blocks everyone
    return {"status": r.status_code}

# GOOD: either use httpx.AsyncClient with await, or (for CPU-bound/blocking
# legacy code) offload to a thread pool with run_in_threadpool.
from starlette.concurrency import run_in_threadpool

@app.get("/good-example")
async def good_example():
    status_code = await run_in_threadpool(blocking_legacy_call)
    return {"status": status_code}
```

D. Edge Cases & Error Handling

- Blocking calls inside async def routes (sync DB driver, requests.get, CPU-heavy loops) silently degrade throughput to near-zero under load — profile with py-spy if latency spikes under concurrency but not in single-request testing
- Pydantic's HttpUrl accepts any scheme by default in some configs; explicitly restrict to http/https or you open the door to file:// or gopher:// SSRF-style tricks later in Ch.3/5
- Returning ORM objects directly without response_model leaks internal columns (password hashes, internal flags) — always map through an explicit *Read schema
- Unhandled exceptions inside a route return a raw 500 with a stack trace unless you add an exception_handler — register one early so production never leaks tracebacks to clients
- Pagination-less list endpoints (GET /monitors/) will eventually try to serialize 100k+ rows into one JSON response — add limit/offset or cursor pagination before this becomes a problem

- CORS misconfiguration (`allow_origins=["*"]` with `allow_credentials=True`) is rejected by browsers and is also a real security footgun — pin explicit origins in production
-

Chapter 2 — SQLAlchemy & Alembic — The Persistence Layer

A. Chapter Objective & Scope

Inputs:

- Validated Pydantic objects from Chapter 1
- Python model class definitions written by the developer

Output Artifacts:

- Durable rows in PostgreSQL (`monitors`, `ping_results`, `users`)
- Versioned migration scripts under `alembic/versions/` that reproduce the schema from scratch

Core Goals:

- One source of truth for schema (the ORM models), never hand-written SQL DDL
- Zero-downtime, reversible schema changes as the product evolves
- Async-safe database access that doesn't block the FastAPI event loop

B. Chapter Flowchart

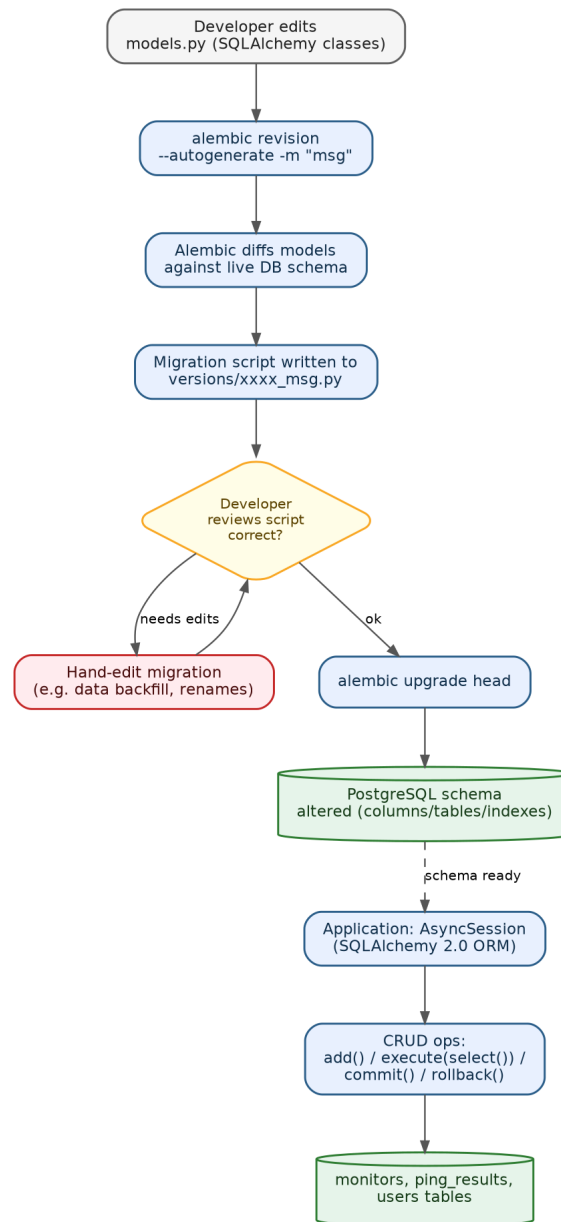


Figure 2.2 — Step-by-step component interaction for Chapter 2.

Mermaid.js source (paste into any Mermaid-compatible renderer / Markdown file to reproduce this diagram):

```
flowchart TB
    Dev["Developer edits models.py (SQLAlchemy classes)"]
    Auto["alembic revision --autogenerate -m 'msg'"]
    Diff["Alembic diffs models against live DB schema"]
    Script["Migration script written to versions/xxxx_msg.py"]
    Review["Developer reviews script: correct?"]
    Fix["Hand-edit migration (data backfill, renames)"]
    Upgrade["alembic upgrade head"]
    PG["PostgreSQL schema altered (columns/tables/indexes)"]
    Session["Application: AsyncSession (SQLAlchemy 2.0 ORM)"]
    CRUD["CRUD ops: add() / execute(select()) / commit() / rollback()"]
    Tables["monitors, ping_results, users tables"]

    Dev --> Auto --> Diff --> Script --> Review
    Review -->|needs edits| Fix --> Review
    Review -->|ok| Upgrade --> PG
    PG -->|schema ready| Session --> CRUD --> Tables
```

C. Code-Side & Technical Implementation

Libraries, Frameworks & Tools

Library / Tool	Exact Role in PingGuard
sqlalchemy>=2.0	ORM + Core; async engine, declarative models, typed Mapped[] columns
asyncpg	The actual async PostgreSQL driver SQLAlchemy's async engine talks to
alembic	Schema migration generation and application, tracks a linear revision history
psycopg2-binary	Sync driver — Alembic itself typically runs migrations synchronously

Data Structures & Schemas

Models use SQLAlchemy 2.0's typed Mapped[] style, which gives you IDE autocomplete and mypy-checkable columns instead of untyped Column() objects.

```

from sqlalchemy import String, Integer, DateTime, ForeignKey, Enum as SAEnum
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship
from datetime import datetime

class Base(DeclarativeBase):
    pass

class Monitor(Base):
    __tablename__ = "monitors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120))
    url: Mapped[str] = mapped_column(String(2048))
    check_interval_seconds: Mapped[int] = mapped_column(Integer, default=60)
    status: Mapped[str] = mapped_column(String(20), default="pending")
    last_checked_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    next_check_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), index=True)

    results: Mapped[list["PingResult"]] = relationship(back_populates="monitor",
                                                    cascade="all, delete-orphan")

class PingResult(Base):
    __tablename__ = "ping_results"

    id: Mapped[int] = mapped_column(primary_key=True)
    monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="CASCADE"), index=True)
    status_code: Mapped[int | None]
    latency_ms: Mapped[int | None]
    error: Mapped[str | None] = mapped_column(String(500))
    checked_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), index=True)

    monitor: Mapped["Monitor"] = relationship(back_populates="results")

```

Execution Pipeline

- 1 Developer writes/edits a Mapped[] class in models.py
- 2 `alembic revision --autogenerate -m "add ping\_results table"` connects to the DB, reflects its current schema, diffs it against Base.metadata, and writes a script
- 3 Developer opens the generated script and reviews the auto-detected op.create_table / op.add_column calls — autogenerate cannot detect every intent (e.g. column renames look like drop+add and will destroy data unless hand-corrected)
- 4 `alembic upgrade head` runs every unapplied revision in order inside a transaction
- 5 Alembic records the applied revision id in its own alembic_version table so it never re-applies or loses track of state, even across deploys
- 6 Application code opens an AsyncSession per request/task and issues typed queries via select(Monitor).where(...) rather than string SQL

Key Code Snippets

Async engine + session factory

```

from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker

DATABASE_URL = "postgresql+asyncpg://user:pass@db:5432/pingguard"

engine = create_async_engine(DATABASE_URL, pool_size=10, max_overflow=20, pool_pre_ping=True)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

```

alembic/env.py — wiring async engine into migrations

```
from alembic import context
from myapp.models import Base
from myapp.db import DATABASE_URL

config = context.config
target_metadata = Base.metadata

def run_migrations_online():
    # Alembic runs migrations synchronously even in an async app;
    # use the sync driver (psycopg2) here, asyncpg for the running app.
    sync_url = DATABASE_URL.replace("+asyncpg", "+psycopg2")
    connectable = create_engine(sync_url)
    with connectable.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)
        with context.begin_transaction():
            context.run_migrations()
```

D. Edge Cases & Error Handling

- Autogenerate cannot distinguish 'rename column' from 'drop column + add column' — always hand-edit renames or you silently lose data on upgrade
- Running migrations from multiple app instances simultaneously on deploy causes lock contention or duplicate-application races — run alembic upgrade as a single pre-deploy step (init container / release phase), never inside every replica's startup hook
- `expire_on_commit=False` (used above) avoids a surprise extra `SELECT` after commit, but means in-memory objects can go stale if another session mutates the same row — `refresh()` explicitly when you need the latest state
- N+1 query patterns (looping over monitors and lazily accessing `.results` per monitor) silently generate one query per row — use `selectinload()/joinedload()` eagerly
- Connection pool exhaustion: async workers + Celery workers + Beat all opening their own pools against the same Postgres instance can exceed `max_connections` — size pools deliberately and consider PgBouncer in front of Postgres at scale
- `DateTime` columns without `timezone=True` store naive datetimes — comparing `next_check_at` across a DST transition or multi-region deploy causes silently wrong scheduling

Chapter 3 — Celery & Redis — Distributed Task Execution

A. Chapter Objective & Scope

Inputs:

- Task invocations: `execute_ping.delay(monitor_id, url)` from Beat or the API
- Redis connection acting as both broker (task queue) and, optionally, result backend

Output Artifacts:

- Ping outcomes persisted to `ping_results`
- Retry/backoff behavior on transient failures; permanent failures marked and surfaced

Core Goals:

- Decouple 'deciding what to check' (API/Beat) from 'actually checking it' (workers)
- Scale ping throughput horizontally by adding worker processes, not by changing code
- Survive individual ping failures without losing the task or corrupting shared state

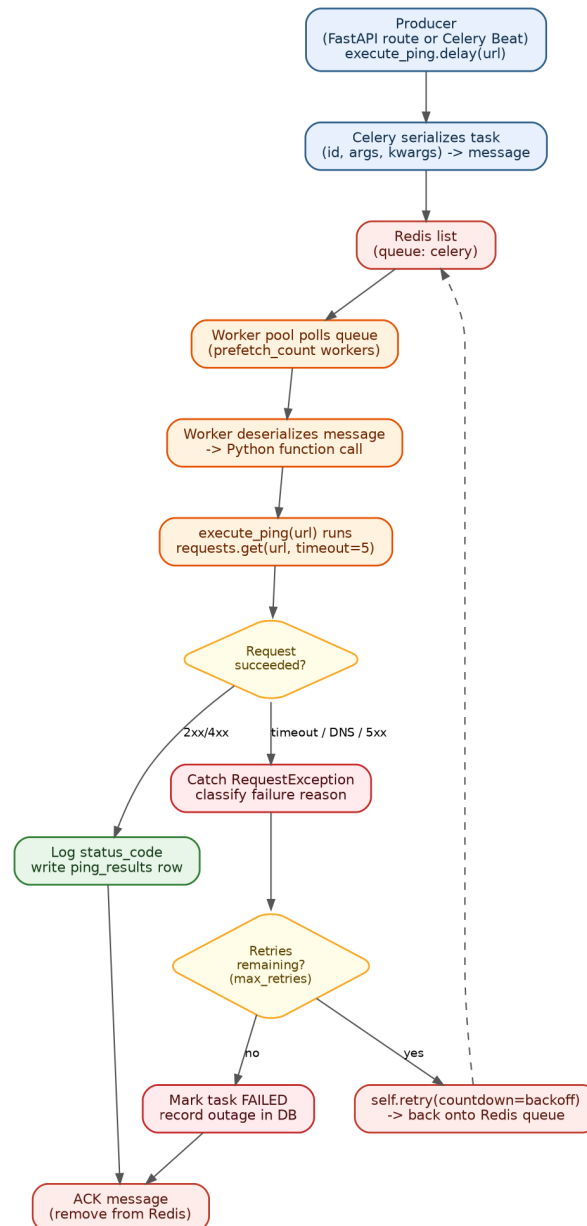
B. Chapter Flowchart

Figure 2.3 — Step-by-step component interaction for Chapter 3.

Mermaid.js source (paste into any Mermaid-compatible renderer / Markdown file to reproduce this diagram):

```

flowchart TB
    Prod["Prod[\"Producer (FastAPI route or Celery Beat): execute_ping.delay(url)\"]"]
    Ser["Ser[\"Celery serializes task (id, args, kwargs) -> message\"]"]
    Redis["Redis[\"Redis list (queue: celery)\"]"]
    Pool["Pool[\"Worker pool polls queue (prefetch_count workers)\"]"]
    Deser["Deser[\"Worker deserializes message -> Python function call\"]"]
    Exec["Exec[\"execute_ping(url) runs requests.get(url, timeout=5)\"]"]
    OK["OK{\"Request succeeded?\"}"]
    Success["Success[\"Log status_code, write ping_results row\"]"]
    Fail["Fail[\"Catch RequestException, classify failure reason\"]"]
    Retry["Retry{\"Retries remaining? (max_retries)\"}"]
    Requeue["Requeue[\"self.retry(countdown=backoff) -> back onto Redis queue\"]"]
    Dead["Dead[\"Mark task FAILED, record outage in DB\"]"]
    Ack["Ack[\"ACK message (remove from Redis)\"]"]

    Prod --> Ser --> Redis --> Pool --> Deser --> Exec --> OK
    OK -->|2xx/4xx| Success --> Ack
    OK -->|timeout / DNS / 5xx| Fail --> Retry
    Retry -->|yes| Requeue --> Redis
    Retry -->|no| Dead --> Ack

```

C. Code-Side & Technical Implementation

Libraries, Frameworks & Tools

Library / Tool	Exact Role in PingGuard
celery[redis]	Task queue framework; produces/consumes messages, manages retries
redis	Python client Celery uses under the hood to talk to the Redis broker
requests	Synchronous HTTP client used inside worker tasks (workers are sync by default)

Data Structures & Schemas

A Celery task message is itself a small JSON envelope. You rarely construct it by hand, but knowing its shape explains a lot of Celery's behavior (retries, idempotency keys, routing).

```

{
  "id": "3f2a9e1e-...-uuid",
  "task": "app.tasks.execute_ping",
  "args": [42, "https://example.com"],
  "kwargs": {},
  "retries": 0,
  "eta": null,
  "expires": null,
  "queue": "pings"
}

```

Execution Pipeline

- 1 Producer calls `execute_ping.delay(monitor_id, url)` (sugar for `.apply_async`)
- 2 Celery serializes the call into the message envelope above and `RPUShes` it onto a Redis list acting as the named queue
- 3 Each worker process runs an event loop of its own (`prefork/gevent/eventless`) that `BLPOPs` from that list — whichever free worker pops the message first executes it

- 4 The task body runs synchronously inside the worker: a plain `requests.get` with an explicit timeout, wrapped in `try/except` for the specific exception classes you care about
- 5 On success, the result (`status_code`, `latency`, `timestamp`) is written straight to Postgres via a sync SQLAlchemy session — not the async one used by FastAPI
- 6 On a transient failure (timeout, connection reset), the task calls `self.retry()` with exponential backoff instead of failing outright
- 7 After `max_retries` is exhausted, the task is marked FAILED and an outage row is written so the failure is visible in the API, not just in worker logs

Key Code Snippets

Celery app configuration

```
from celery import Celery

celery_app = Celery(
    "pingguard",
    broker="redis://redis:6379/0",
    backend="redis://redis:6379/1",  # separate DB index from broker
)

celery_app.conf.update(
    task_serializer="json",
    result_serializer="json",
    accept_content=["json"],
    task_acks_late=True,           # ack only after the task finishes, not on receipt
    worker_prefetch_multiplier=1, # avoid one slow worker hoarding many tasks
    task_time_limit=15,           # hard kill if a task somehow runs past 15s
    task_soft_time_limit=10,      # raise a catchable exception at 10s
)
```

The ping task with retry/backoff

```
import requests
from celery import shared_task
from celery.exceptions import MaxRetriesExceededError

@shared_task(bind=True, max_retries=3, name="app.tasks.execute_ping")
def execute_ping(self, monitor_id: int, url: str):
    try:
        response = requests.get(url, timeout=5, allow_redirects=True)
        save_result(monitor_id, status_code=response.status_code, error=None)
    except requests.Timeout:
        raise self.retry(exc=None, countdown=2 ** self.request.retries)
    except requests.ConnectionError as exc:
        try:
            raise self.retry(exc=exc, countdown=2 ** self.request.retries)
        except MaxRetriesExceededError:
            save_result(monitor_id, status_code=None, error="connection_failed")
```

D. Edge Cases & Error Handling

- `task_acks_late=False` (the default) means a task ACKed on receipt is LOST if the worker process crashes mid-execution — for anything you can't afford to silently drop, set `acks_late=True` and make the task idempotent (safe to run twice)

- Idempotency matters here specifically because `acks_late` + a crash right after execution but before the ack can cause the same ping to be recorded twice — use an idempotency key (e.g. `upsert on (monitor_id, checked_at)` truncated to the check interval)
 - `worker_prefetch_multiplier > 1` with unevenly slow tasks causes head-of-line blocking: a worker can hold 4 tasks while only running one, starving other workers that are idle
 - SSRF: users supply arbitrary URLs to ping. Without egress restrictions, an attacker can point a monitor at `http://169.254.169.254/` (cloud metadata endpoint) or internal-only services — validate/deny private IP ranges and cloud metadata addresses at task time
 - Retry storms: if a widely-monitored site goes down, thousands of monitors retry in near lockstep — always use jittered exponential backoff, never fixed-interval retry
 - Redis is an in-memory broker; without AOF/RDB persistence configured, queued-but-not-yet-executed tasks are lost on a Redis restart — acceptable for pings (next tick recreates them) but not for anything you can't afford to silently skip once
 - Result backend growth: if you don't set `ignore_result=True` on fire-and-forget tasks, Redis accumulates a result entry per task forever until TTL/eviction is configured
-

Chapter 4 — Celery Beat — The Scheduling Heartbeat

A. Chapter Objective & Scope

Inputs:

- The current wall-clock time, on a fixed tick (e.g. every 15–60 seconds)
- `monitors.next_check_at` column in PostgreSQL

Output Artifacts:

- A batch of `execute_ping` tasks pushed to Redis, one per due monitor
- Updated `next_check_at` values so the same monitor isn't re-enqueued next tick

Core Goals:

- Decide *when* to check something — a decision totally separate from *how* to check it
- Support a dynamic, database-driven set of monitors rather than a fixed, hardcoded schedule (this is why we do NOT use Celery's static `beat_schedule` dict for per-monitor entries — monitors are created/deleted at runtime by users)
- Never enqueue the same due monitor twice, and never miss one, even across restarts

B. Chapter Flowchart

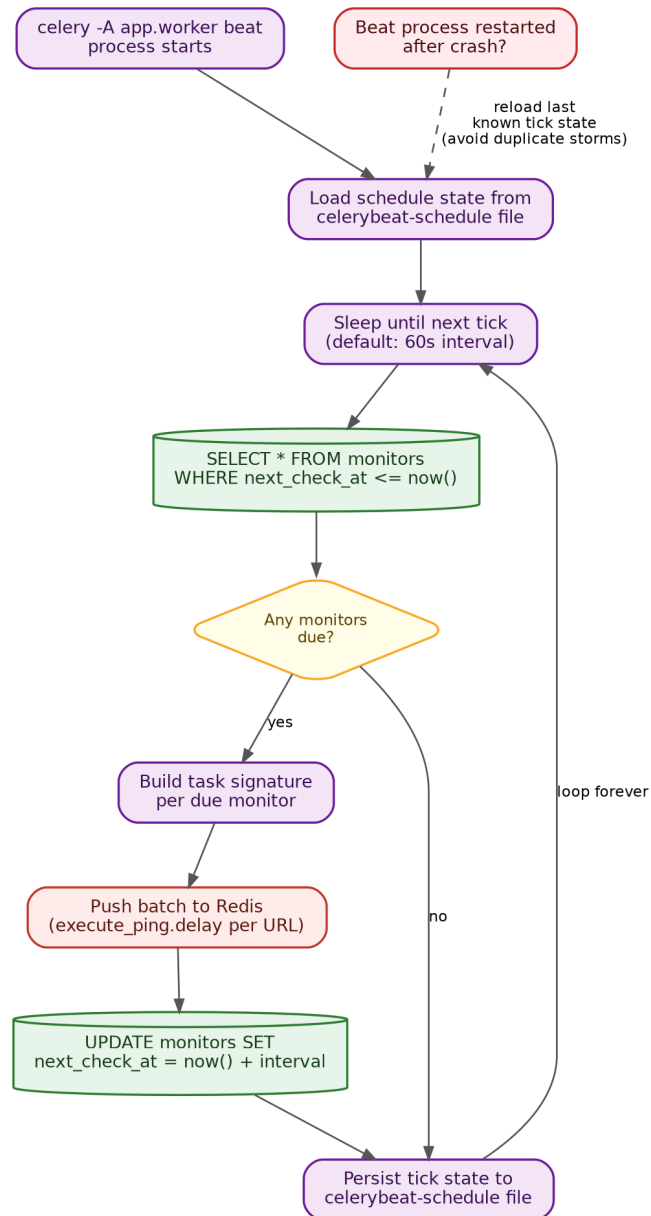


Figure 2.4 — Step-by-step component interaction for Chapter 4.

Mermaid.js source (paste into any Mermaid-compatible renderer / Markdown file to reproduce this diagram):

```

flowchart TB
    Start["celery -A app.worker beat process starts"]
    Load["Load schedule state from celerybeat-schedule file"]
    Tick["Sleep until next tick (default: 60s interval)"]
    Query["SELECT * FROM monitors WHERE next_check_at <= now()"]
    Any{"Any monitors due?"}
    Batch["Build task signature per due monitor"]
    Push["Push batch to Redis (execute_ping.delay per URL)"]
    Update["UPDATE monitors SET next_check_at = now() + interval"]
    Persist["Persist tick state to celerybeat-schedule file"]
    Crash["Beat process restarted after crash?"]

    Start --> Load --> Tick --> Query --> Any
    Any -->|yes| Batch --> Push --> Update --> Persist
    Any -->|no| Persist
    Persist -->|loop forever| Tick
    Crash -->|reload last known tick state, avoid duplicate storms| Load

```

C. Code-Side & Technical Implementation

Libraries, Frameworks & Tools

Library / Tool	Exact Role in PingGuard
celery[redis]	Beat is a mode of the same Celery package (`celery beat`), not a separate lib
django-celery-beat is NOT used here	That package assumes Django's ORM; we drive the sweep query directly against SQLAlchemy instead

Data Structures & Schemas

Rather than one static `beat_schedule` entry per URL (which would require restarting Beat every time a user adds a monitor), PingGuard uses a single periodic 'sweep' task that queries the database for whatever is due right now.

```

# celery_app.conf.beat_schedule — ONE static entry, not one per monitor
celery_app.conf.beat_schedule = {
    "sweep-due-monitors": {
        "task": "app.tasks.sweep_due_monitors",
        "schedule": 15.0, # seconds; tune based on your minimum check_interval_seconds
    },
}

```

Execution Pipeline

- 1 `celery -A app.worker beat` starts a single, dedicated scheduler process
- 2 Beat loads its last-known tick state from the `celerybeat-schedule` file (a local shelve/dbm file) so a restart doesn't cause an immediate duplicate fire
- 3 On each tick, it fires the one registered periodic task: `sweep_due_monitors`
- 4 That task (running on a normal Celery worker, not inside Beat itself) runs a single query: `SELECT * FROM monitors WHERE next_check_at <= now()`
- 5 For each row returned, it calls `execute_ping.delay(monitor.id, monitor.url)`
- 6 It then updates `next_check_at = now() + check_interval_seconds` for every row it just enqueued, in the same transaction as the read, using `SELECT ... FOR UPDATE SKIP LOCKED`
- 7 Beat goes back to sleep until the next tick

Key Code Snippets

The sweep task — concurrency-safe claim pattern

```
from sqlalchemy import select
from datetime import datetime, timedelta, timezone

@shared_task(name="app.tasks.sweep_due_monitors")
def sweep_due_monitors():
    with SyncSessionLocal() as db:
        now = datetime.now(timezone.utc)
        # FOR UPDATE SKIP LOCKED lets multiple sweep runs overlap safely if one
        # ever runs long -- each row can only be claimed by one sweep at a time.
        stmt = (
            select(Monitor)
            .where(Monitor.next_check_at <= now)
            .with_for_update(skip_locked=True)
        )
        due = db.execute(stmt).scalars().all()
        for monitor in due:
            execute_ping.delay(monitor.id, monitor.url)
            monitor.next_check_at = now + timedelta(seconds=monitor.check_interval_seconds)
        db.commit()
```

Running exactly one Beat instance (critical!)

```
# docker-compose.yml excerpt — Beat MUST be singleton-replicated.
# Two Beat processes reading the same schedule file will double-fire every tick.
services:
  scheduler:
    build: .
    command: celery -A app.worker.celery_app beat --loglevel=info
    deploy:
      replicas: 1          # never scale this service horizontally
      depends_on: [redis, db]
```

D. Edge Cases & Error Handling

- Running more than one Beat replica is the single most common production bug in this architecture — every tick gets duplicated, doubling ping traffic and result rows; use a distributed lock (Redis SETNX) or your orchestrator's singleton-pod guarantee if you need HA
- Clock drift between the Beat host and the database host can cause systematically early or late sweeps — run NTP-synced hosts and prefer the database's now() over app-server time for the WHERE clause, as this query already does
- Without SELECT ... FOR UPDATE SKIP LOCKED, an overlapping sweep (e.g. one that runs long because the DB was momentarily slow) can double-claim the same monitor and enqueue it twice
- A tick interval shorter than your smallest check_interval_seconds wastes queries for no benefit; a tick interval longer than it means monitors silently check less often than configured — keep tick interval well below the minimum allowed check_interval_seconds
- The celerybeat-schedule state file lives on local disk inside the container — on a container replace (not just restart) this file is lost unless it's on a mounted volume, causing an immediate duplicate fire right after deploy
- A large fleet (millions of monitors) will eventually make the single sweep query itself slow — partition by next_check_at with a covering index, or shard monitors across multiple sweep tasks/queues by hash range

Chapter 5 — httpx & Network Resilience

A. Chapter Objective & Scope

Inputs:

- A target URL and a bounded time budget

Output Artifacts:

- A classified outcome: UP / DEGRADED / DOWN / and a specific failure reason
- latency_ms for successful checks (used for trend graphs, SLA reporting)

Core Goals:

- Never let one unreachable site hang a worker indefinitely
- Distinguish 'the site is down' from 'the site is fine but returned an error page' from 'we couldn't even resolve the domain' — these need different alerting and messaging

B. Chapter Flowchart

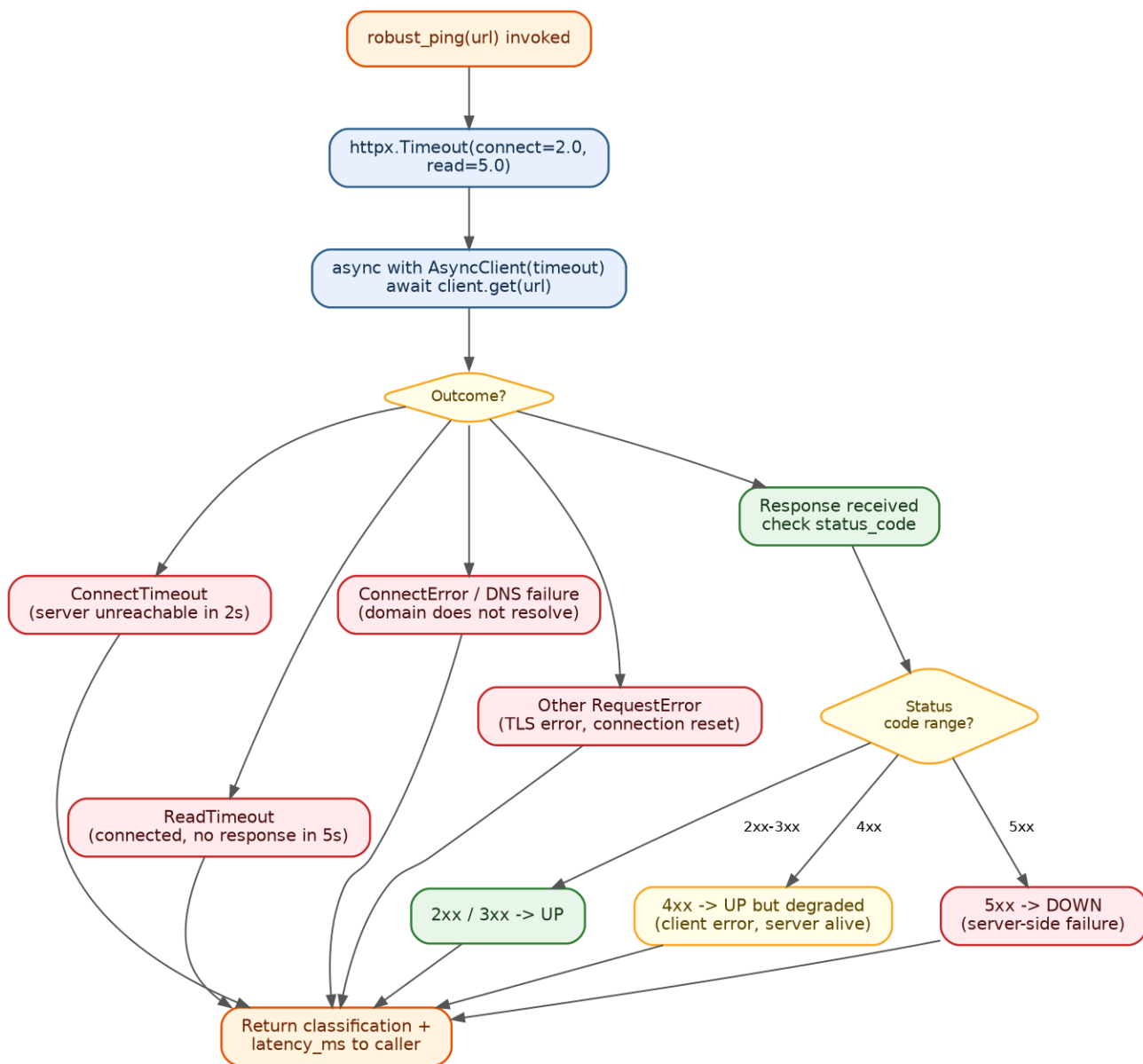


Figure 2.5 — Step-by-step component interaction for Chapter 5.

Mermaid.js source (paste into any Mermaid-compatible renderer / Markdown file to reproduce this diagram):

```

flowchart TB
    Call["robust_ping(url) invoked"]
    Timeout["httpx.Timeout(connect=2.0, read=5.0)"]
    Open["async with AsyncClient(timeout): await client.get(url)"]
    Outcome["Outcome?"]
    ConnTO["ConnectTimeout (server unreachable in 2s)"]
    ReadTO["ReadTimeout (connected, no response in 5s)"]
    DNS["ConnectError / DNS failure (domain does not resolve)"]
    Other["Other RequestError (TLS error, connection reset)"]
    Success["Response received, check status_code"]
    Code["Status code range?"]
    Up["2xx / 3xx -> UP"]
    Degraded["4xx -> UP but degraded (client error, server alive)"]
    Down["5xx -> DOWN (server-side failure)"]
    Classify["Return classification + latency_ms to caller"]

    Call --> Timeout --> Open --> Outcome
    Outcome --> ConnTO --> Classify
    Outcome --> ReadTO --> Classify
    Outcome --> DNS --> Classify
    Outcome --> Other --> Classify
    Outcome --> Success --> Code
    Code -->|2xx-3xx| Up --> Classify
    Code -->|4xx| Degraded --> Classify
    Code -->|5xx| Down --> Classify

```

C. Code-Side & Technical Implementation

Libraries, Frameworks & Tools

Library / Tool	Exact Role in PingGuard
httpx	Modern HTTP client with first-class async support and granular timeout controls
certifi	CA bundle httpx uses for TLS verification (pin/update deliberately)

Data Structures & Schemas

The result of a ping is modeled as a small tagged union so callers can't forget to handle a failure branch.

```

from dataclasses import dataclass
from enum import Enum

class PingOutcome(str, Enum):
    UP = "up"
    DEGRADED = "degraded" # reached the server, got a 4xx
    DOWN = "down" # reached the server, got a 5xx
    UNREACHABLE = "unreachable" # never got a response at all

@dataclass
class PingResultDTO:
    outcome: PingOutcome
    status_code: int | None
    latency_ms: int | None
    error_detail: str | None

```

Execution Pipeline

- 1 Caller sets an explicit `httpx.Timeout` with separate `connect/read/write/pool` phases — a single flat `timeout=5` hides which phase actually failed
- 2 `AsyncClient` opens the connection (or reuses one from its keep-alive pool)

- 3 Each distinct exception type is caught separately, not with a bare except: — this is what lets you tell 'DNS never resolved' apart from 'server took too long'
- 4 On a response, `status_code` is bucketed: 2xx/3xx = UP, 4xx = DEGRADED (the server answered, so it's alive — the resource or request itself is wrong), 5xx = DOWN
- 5 Latency is measured around the `await` call, not around the whole function, so DNS caching warm-up doesn't pollute the very first measurement's meaning
- 6 The classified `PingResultDTO` is what Chapter 3's task hands to the persistence layer — the worker task never touches raw exceptions itself

Key Code Snippets

Full classification function

```
import httpx, time

async def robust_ping(url: str) -> PingResultDTO:
    timeout = httpx.Timeout(connect=2.0, read=5.0, write=5.0, pool=2.0)
    start = time.monotonic()
    try:
        async with httpx.AsyncClient(timeout=timeout, follow_redirects=True) as client:
            response = await client.get(url)
            latency_ms = int((time.monotonic() - start) * 1000)
            if response.status_code < 400:
                outcome = PingOutcome.UP
            elif response.status_code < 500:
                outcome = PingOutcome.DEGRADED
            else:
                outcome = PingOutcome.DOWN
            return PingResultDTO(outcome, response.status_code, latency_ms, None)
    except httpx.ConnectTimeout:
        return PingResultDTO(PingOutcome.UNREACHABLE, None, None, "connect_timeout")
    except httpx.ReadTimeout:
        return PingResultDTO(PingOutcome.UNREACHABLE, None, None, "read_timeout")
    except httpx.ConnectError as exc:
        return PingResultDTO(PingOutcome.UNREACHABLE, None, None, f"connect_error:{exc}")
    except httpx.RequestError as exc:
        return PingResultDTO(PingOutcome.UNREACHABLE, None, None, f"request_error:{exc}")
```

Guarding against SSRF at the network layer

```
import ipaddress, socket

BLOCKED_NETWORKS = [ipaddress.ip_network(n) for n in (
    "127.0.0.0/8", "10.0.0.0/8", "172.16.0.0/12", "192.168.0.0/16",
    "169.254.0.0/16", # link-local / cloud metadata (AWS, GCP, Azure)
)]

def resolves_to_private_ip(hostname: str) -> bool:
    for family, _, _, sockaddr in socket.getaddrinfo(hostname, None):
        ip = ipaddress.ip_address(sockaddr[0])
        if any(ip in net for net in BLOCKED_NETWORKS):
            return True
    return False

# Call this once at monitor-creation time AND again at ping time (DNS can
# change between the two -- this is a classic TOCTOU/rebinding attack).
```

D. Edge Cases & Error Handling

- A flat timeout=N hides whether the failure was DNS, connect, or read — always split into connect/read/write/pool so operators can tell 'server is slow' from 'server is dead'
- follow_redirects=True (needed for real-world URLs) means the effective destination can differ from the configured one — log the final response.url alongside the original
- DNS rebinding: a URL can resolve to a public IP at monitor-creation time and a private/metadata IP later — re-validate the resolved IP at ping time, not just at creation time
- Very large response bodies from a misbehaving target can exhaust worker memory — stream the response and cap bytes read (client.stream() + a byte counter) rather than materializing the full body when you only need the status code
- TLS certificate errors (expired, self-signed, hostname mismatch) surface as httpx.ConnectError wrapping an SSL error — don't swallow this into a generic 'unreachable' if you want to alert 'certificate expired' differently from 'server down'
- Keep-alive connection pools reused across many different hosts inside one AsyncClient can leak file descriptors under high monitor counts — scope one client per worker task or per batch, not one global client shared unbounded across the process lifetime

Chapter 6 — Docker & Docker Compose — Orchestration

A. Chapter Objective & Scope

Inputs:

- Dockerfile(s), docker-compose.yml, .env secrets

Output Artifacts:

- Five running, networked containers reachable by service name

Core Goals:

- Reproduce the exact same runtime on any machine (dev laptop, CI, staging)
- Start dependency-ordered services without five manual terminal windows
- Isolate each concern (API, worker, scheduler, broker, database) as its own process with its own resource limits and restart policy

B. Chapter Flowchart

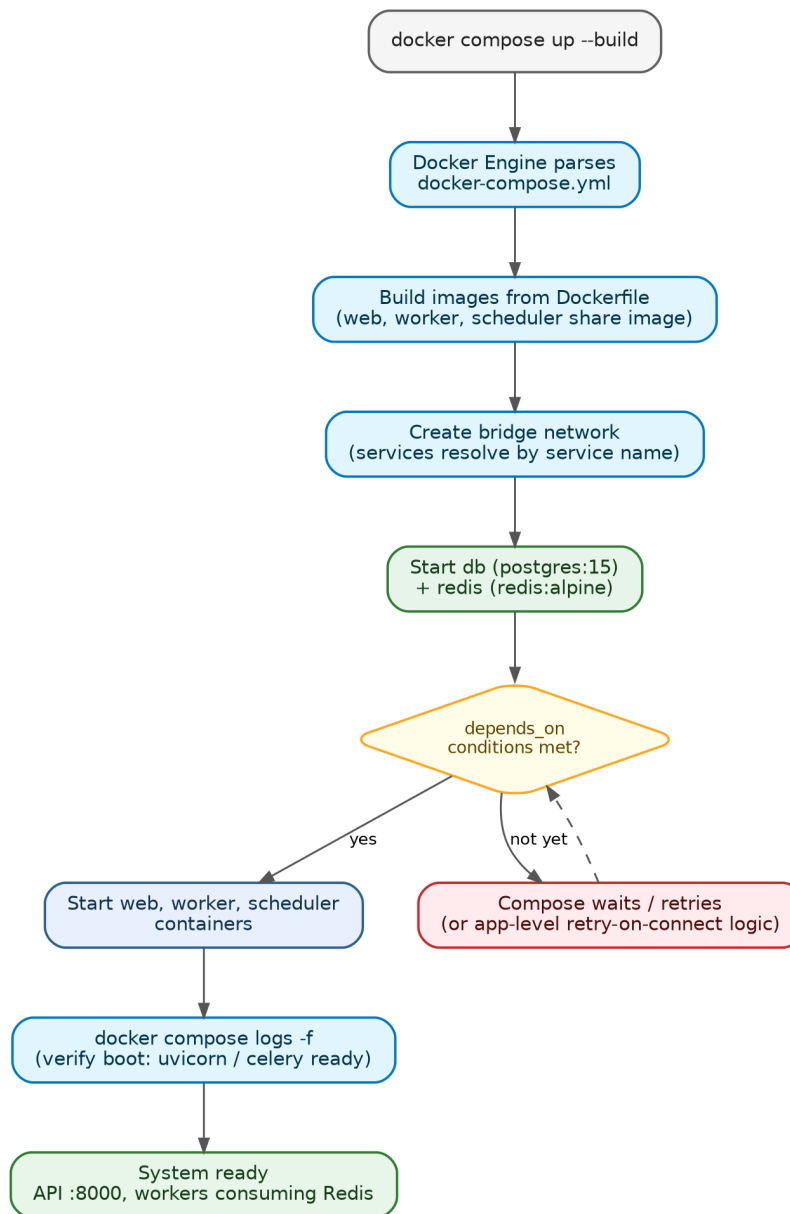


Figure 2.6 — Step-by-step component interaction for Chapter 6.

Mermaid.js source (paste into any Mermaid-compatible renderer / Markdown file to reproduce this diagram):

```

flowchart TB
    Cmd["docker compose up --build"]
    Parse["Docker Engine parses docker-compose.yml"]
    Build["Build images from Dockerfile (web, worker, scheduler share image)"]
    Net["Create bridge network (services resolve by service name)"]
    DbUp["Start db (postgres:15) + redis (redis:alpine)"]
    Healthy{"depends_on conditions met?"}
    Wait["Compose waits / retries (or app-level retry-on-connect logic)"]
    AppUp["Start web, worker, scheduler containers"]
    Logs["docker compose logs -f (verify boot: uvicorn / celery ready)"]
    Ready["System ready: API :8000, workers consuming Redis"]

    Cmd --> Parse --> Build --> Net --> DbUp --> Healthy
    Healthy -->|not yet| Wait --> Healthy
    Healthy -->|yes| AppUp --> Logs --> Ready

```

C. Code-Side & Technical Implementation

Libraries, Frameworks & Tools

Library / Tool	Exact Role in PingGuard
Docker Engine	Builds images and runs containers
Docker Compose v2 (docker compose)	Declarative multi-container orchestration

Data Structures & Schemas

web, worker, and scheduler all share the same image (same Dockerfile, same installed dependencies) — only the container command differs. This avoids dependency drift between the API and the workers that execute its background tasks.

```

# Dockerfile — shared by web / worker / scheduler
FROM python:3.12-slim
WORKDIR /app
COPY pyproject.toml poetry.lock ./
RUN pip install --no-cache-dir poetry && poetry config virtualenvs.create false \
    && poetry install --no-root --only main
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Execution Pipeline

- 1 ``docker compose up --build`` parses `docker-compose.yml` and resolves the service dependency graph from `depends_on` entries
- 2 Images are built (or pulled, for db/redis) and cached by layer — unchanged layers (system packages, dependencies) are reused across rebuilds
- 3 A bridge network is created; every service can reach every other by its service name as a hostname (web reaches Postgres at db:5432, not localhost)
- 4 db and redis start first; `depends_on` with condition: `service_healthy` makes Compose wait for their healthchecks, not just their process start, before starting dependents
- 5 web, worker, and scheduler start once their dependencies report healthy

- 6 ``docker compose logs -f`` tails all containers together, interleaved and color-coded by service, which is how you verify unicorn bound its port and Celery workers connected

Key Code Snippets

Production-grade docker-compose.yml

```
version: "3.8"

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: pingguard
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: pingguard
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U pingguard"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  web:
    build: .
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000
    ports: ["8000:8000"]
    env_file: .env
    depends_on:
      db: {condition: service_healthy}
      redis: {condition: service_healthy}

  worker:
    build: .
    command: celery -A app.worker.celery_app worker --loglevel=info --concurrency=8
    env_file: .env
    depends_on:
      db: {condition: service_healthy}
      redis: {condition: service_healthy}
    deploy:
      replicas: 2 # horizontally scalable

  scheduler:
    build: .
    command: celery -A app.worker.celery_app beat --loglevel=info
    env_file: .env
    depends_on:
      db: {condition: service_healthy}
      redis: {condition: service_healthy}
    deploy:
      replicas: 1 # NEVER scale this one – see Chapter 4

volumes:
  pgdata:
```

D. Edge Cases & Error Handling

- `depends_on` without a healthcheck condition only waits for the container process to start, not for Postgres to actually accept connections — web can crash-loop on boot against a still-initializing database unless you add condition: `service_healthy`
 - Baking secrets (DB passwords, API keys) into the image via ARG/ENV in the Dockerfile leaves them recoverable from image layers — inject them at runtime via `env_file/.env` instead, and keep `.env` out of version control
 - Named volumes (`pgdata`) persist across ``docker compose down``; only ``docker compose down -v`` removes them — a routine 'reset everything' during development can accidentally nuke data if `-v` is muscle-memoried in without thinking
 - Scaling the scheduler service (`replicas > 1`) causes the duplicate-Beat problem from Chapter 4 — Compose's ``deploy.replicas`` is honored under ``docker compose up --scale``, so make this a documented, enforced constraint, not just a comment
 - Without explicit resource limits (`mem_limit / deploy.resources.limits`), a runaway worker or a memory leak in a long-lived `AsyncClient` can starve the host and take down sibling containers, including the database
 - Rolling restarts of the web service between requests need graceful shutdown handling (`SIGTERM` → stop accepting new connections → finish in-flight requests) or you'll see intermittent connection-reset errors on every deploy under real traffic
-

3. End-to-End Execution Roadmap

Build in this exact order. Each milestone is independently runnable and verifiable before you add the next layer — do not start Chapter 3 until Chapter 2's migrations apply cleanly against a running Postgres instance.

3.0 Project Scaffold

Directory layout

```
pingguard/
├── app/
│   ├── main.py          # FastAPI app + routes (Ch.1)
│   ├── schemas.py       # Pydantic models (Ch.1)
│   ├── models.py        # SQLAlchemy models (Ch.2)
│   ├── db.py            # engine + session factories (Ch.2)
│   ├── worker.py        # Celery app config (Ch.3/4)
│   ├── tasks.py         # execute_ping, sweep_due_monitors (Ch.3/4)
│   └── net.py           # robust_ping, SSRF guard (Ch.5)
├── alembic/
│   ├── env.py
│   └── versions/
├── Dockerfile
├── docker-compose.yml
├── pyproject.toml
├── .env
└── alembic.ini
```

pyproject.toml — dependency manifest

```
[tool.poetry.dependencies]
python = "^3.12"
fastapi = "^0.115"
uvicorn = {extras = ["standard"], version = "^0.32"}
pydantic = "^2.9"
pydantic-settings = "^2.5"
sqlalchemy = "^2.0"
asynccpg = "^0.30"
psycopg2-binary = "^2.9"
alembic = "^1.13"
celery = {extras = ["redis"], version = "^5.4"}
redis = "^5.1"
requests = "^2.32"
httpx = "^0.27"
certifi = "^2024.8.30"
```

3.1 Milestone Sequence

Milestone 1 — Chapter 1 (API skeleton)

- 1 poetry install
- 2 Write app/schemas.py and a minimal app/main.py with an in-memory list (no DB yet)
- 3 uvicorn app.main:app --reload
- 4 Verify: `curl -X POST localhost:8000/monitors/ -d '{"name":"t","url":"https://x.com"}' -H 'Content-Type: application/json'` returns 201

- 5 Verify: open `http://localhost:8000/docs` and confirm the interactive schema matches `MonitorCreate/MonitorRead` exactly

Milestone 2 — Chapter 2 (Persistence)

- 1 `docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=dev postgres:15` (local DB for dev)
- 2 Write `app/models.py` and `app/db.py`; wire `get_db()` into `main.py`'s routes
- 3 `alembic init alembic`, then edit `env.py` per the snippet above
- 4 `alembic revision --autogenerate -m "initial schema"`
- 5 `alembic upgrade head`
- 6 Verify: `psql -h localhost -U pingguard -c '\dt'` lists monitors and `ping_results`
- 7 Verify: the same curl from Milestone 1 now persists across an app restart

Milestone 3 — Chapter 3 (Celery + Redis)

- 1 `docker run -d -p 6379:6379 redis:7-alpine` (local broker for dev)
- 2 Write `app/worker.py` (Celery config) and `app/tasks.py` (`execute_ping`)
- 3 `celery -A app.worker.celery_app worker --loglevel=info` (separate terminal)
- 4 Verify: from a Python shell, `execute_ping.delay(1, "https://example.com")` returns immediately and the worker terminal logs the ping completing
- 5 Verify: kill the worker mid-task and confirm `task_acks_late` behavior matches expectations

Milestone 4 — Chapter 4 (Beat scheduling)

- 1 Write `sweep_due_monitors` in `tasks.py` and register it in `beat_schedule`
- 2 `celery -A app.worker.celery_app beat --loglevel=info` (separate terminal, third process)
- 3 Verify: create a monitor with `check_interval_seconds=15`, then watch the worker log a ping within 15–30 seconds with no manual trigger
- 4 Verify: confirm `next_check_at` advances correctly after each sweep (query Postgres directly)

Milestone 5 — Chapter 5 (Resilience)

- 1 Replace the bare `requests.get` in `execute_ping` with `robust_ping`'s classification logic (adapted to sync requests, or move the task to Celery's asyncio-friendly runner)
- 2 Add the SSRF guard (`resolves_to_private_ip`) to monitor creation in Chapter 1's route
- 3 Verify: point a monitor at `http://169.254.169.254/` and confirm creation is rejected
- 4 Verify: point a monitor at a deliberately slow endpoint (`httpbin.org/delay/10`) and confirm it's classified as a timeout, not a hang

Milestone 6 — Chapter 6 (Containerize everything)

- 1 Write the shared Dockerfile and the full `docker-compose.yml` from Chapter 6
- 2 `docker compose up --build`
- 3 Verify: `docker compose ps` shows all five services healthy

- 4 Verify: `docker compose logs -f worker` shows tasks being consumed from the containerized Redis, not a leftover local instance from earlier milestones
- 5 Verify: `docker compose down` && `docker compose up` brings the whole stack back with monitors intact (named volume persistence)

3.2 Beyond Local Development

- Run behind Gunicorn with uvicorn workers (`gunicorn -k uvicorn.workers.UvicornWorker`) for multi-process API concurrency in production, not `--reload` uvicorn
- Move Postgres and Redis to managed services (RDS/Cloud SQL, ElastiCache/Memorystore) so persistence and failover aren't your team's problem to operate
- Run ``alembic upgrade head`` as a one-shot release-phase job, never inside every replica's container endpoint (see Chapter 2 edge cases)
- Add Flower (or Celery's built-in events) for live worker/queue observability, but put it behind authentication — it exposes task arguments, which may include internal URLs
- Instrument with OpenTelemetry across the API → Redis → worker boundary so a single trace ID follows a monitor from creation through its ping result, even across processes
- Autoscale the worker service on queue depth (Redis LLEN on the task queue), not on CPU — ping tasks are I/O-bound and CPU will look idle even when the queue is backing up